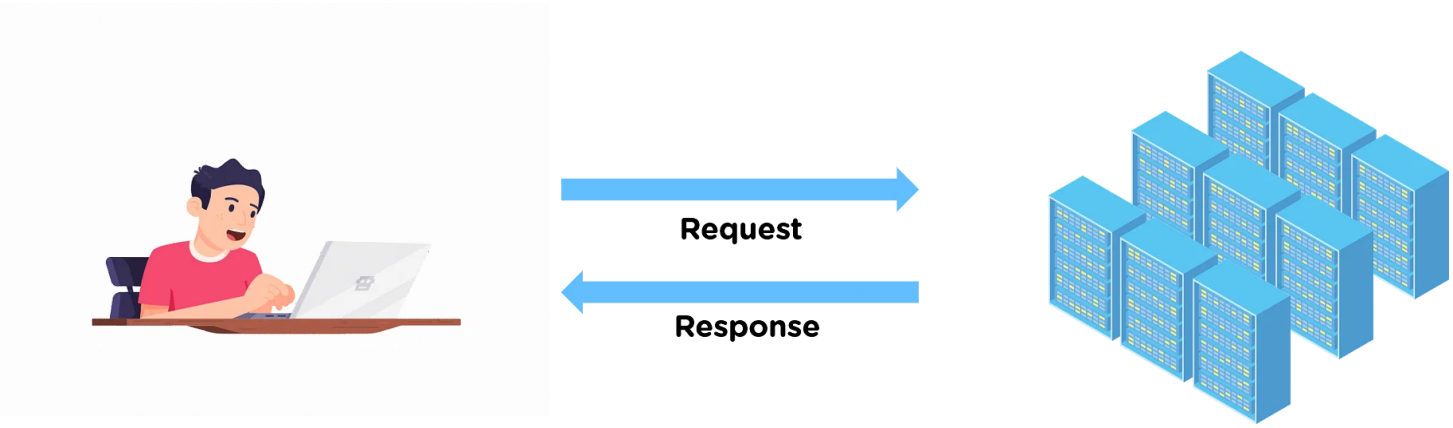
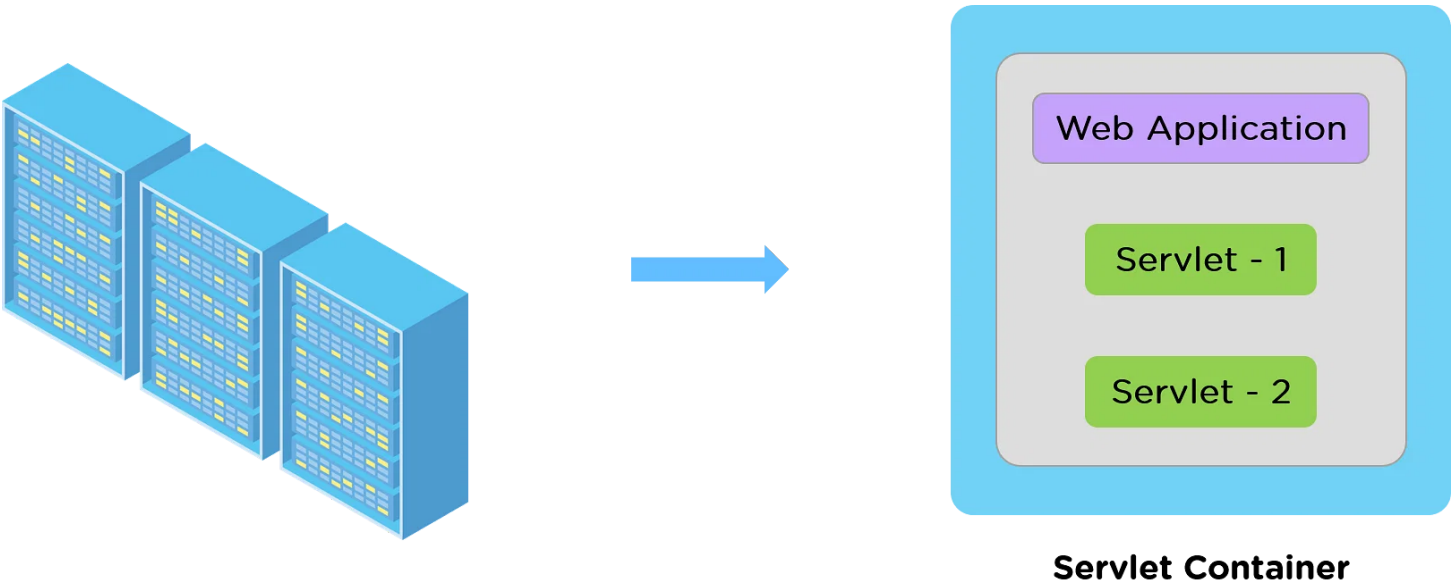
What is a Java Servlet?



We define a Java Servlet or Jakarta Servlet as the technology to design and deploy dynamic web pages using the Java Programming Language. It implements a typical servlet in the client-server architecture, and the Servlet lives on the server-side.



There are several varieties of interfaces and classes available in the Servlet API. Some of them are as follows:

- HTTP Servlet
- Generic Servlet
- Servlet Request

- Servlet Response

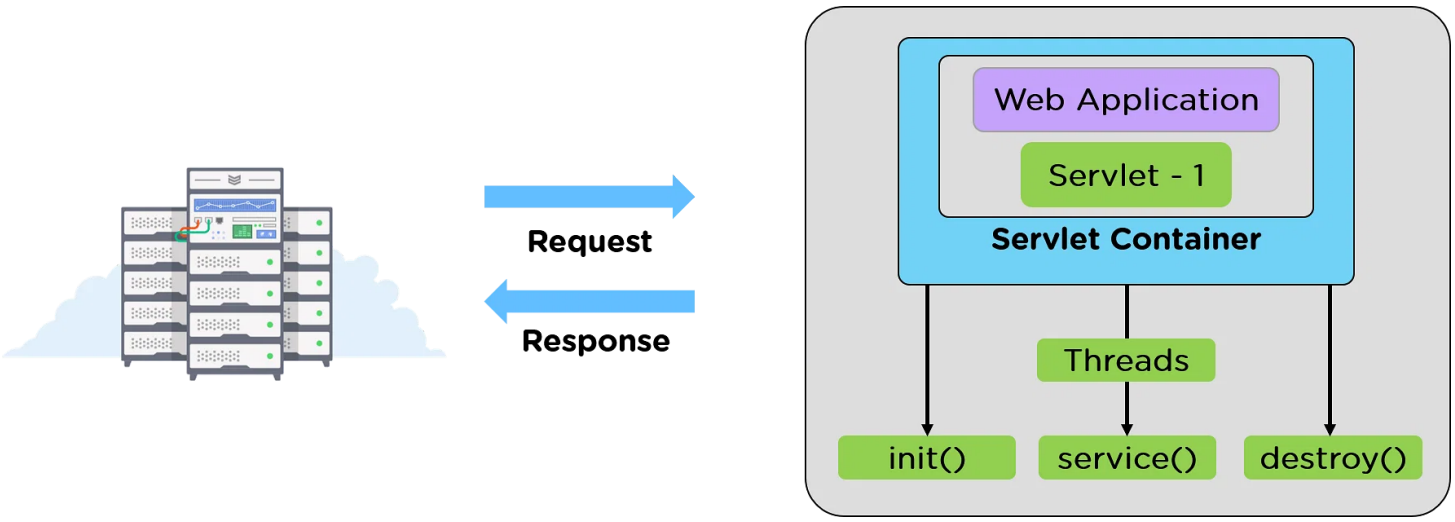
With the definition, interfaces, and classes of Java Servlets discussed, we will now advance into our next topic, learning about the Java Servlet's Architecture.

Become a Skilled Web Developer in Just 9 Months!

Caltech PGP Full Stack Development

EXPLORE PROGRAM

Java Servlet Architecture



We can also consider the software architecture of a servlet as the life cycle of the Java Servlet. To write a Servlet, the user needs first to implement the Servlet Interface, directly or indirectly, using the following import command.

```
import javax.servlet.*;
```

Once the Servlet interface is imported, and we inherit the HTTP Class, we begin with the Java Servlet's life cycle.

In the life cycle of a servlet, we have mainly three stages, which are mentioned below.

- init()
- service()
- destroy()

We call these methods at their respective stages. The methods are resolved by generating the essential threads for the process to get executed.

Next, we will learn the Life-Cycle of a Java Servlet in a detailed manner.

Java Servlet Life-Cycle

The Java Servlet Life cycle includes three stages right from its start to the end until the Garbage Collector clears it. These three stages are described below.

1. init()
2. service()
3. destroy()

1. init()

The init() is the germinating stage of any Java Servlet. When a URL specific to a particular servlet is triggered, the init() method is invoked.

Another scenario when the init() method gets invoked is when the servers are fired up. With every server starting up, the corresponding servlets also get started, and so does the init() method.

One important specialty of the init() method is the init() method only gets invoked once in the entire life cycle of the Servlet, and the init() method will not respond to any of the user's commands.

The init() method Syntax:

```
public void init() throws ServletException {
```

```
//init() method initializing

}
```

2. service()

The service() method is the heart of the life cycle of a Java Servlet. Right after the Servlet's initialization, it encounters the service requests from the client end.

The client may request various services like:

- GET
- PUT
- UPDATE
- DELETE

The service() method takes responsibility to check the type of request received from the client and respond accordingly by generating a new thread or a set of threads per the requirement and implementing the operation through the following methods.

- doGet() for GET
- doPut() for PUT
- doUpdate() for UPDATE
- doDelete() for DELETE

The service() method Syntax:

```
public void service(ServletRequest request, ServletResponse response)

throws ServletException, IOException {

}
```

Become a Skilled Web Developer in Just 9 Months!

Caltech PGP Full Stack Development

EXPLORE PROGRAM

3. destroy()

Like the init() method, the destroy() method is also called only once in the Java Servlet's entire life cycle.

When the destroy() method is called, the Servlet performs the cleanup activities like,

- Halting the current or background threads
- Making a recovery list of any related data like cookies to Disk.

After that, the Servlet is badged, ready for the Garbage collector to have it cleared.

The destroy() method Syntax:

```
public void destroy() {

//destroy() method finalizing

}
```

Finishing with the life cycle of the Java Servlet, we are now advancing further, where we encounter the Java Servlet features.

Learn top skills demanded in the industry, including Angular, Spring Boot, Hibernate, Servlets, and JSPs, as well as MVC, web services, and SOA to build highly web scalable apps with the Full Stack Java Developer Masters Program.

Java Servlet Features

The Java Servlets carry over the features of Java Programming Language. The key features offered by the Java Servlets are as follows.

- Portable
- Efficient
- Scalable
- Robust

1. Portable

As discussed above, the servlets feature the same Portable nature as the Java Programming Language. The Servlet program designed in one Operating System's Platform can be run in a different Operating System Platform with ease.

2. Efficient

The Java Servlet, by its nature, provides an instantaneous response to the client's request. Also, it can be portable and perform in every environmental condition regardless of the operating system platform.

3. Scalable

We consider Java Servlets to be highly scalable. Servlets use completely lightweight threads for the processes and can simultaneously handle multiple client requests by generating various threads.

4. Robust

The Java Servlets are best known for their robust operating procedures. The servlets extend the features of Java, which include.

1. Java Security Manager
2. Java Garbage Collector
3. Exception Handling.

Implementation of these advanced features makes Java Servlets resilient to any security threats to a decent extent. Also, the garbage collector takes care of memory management and eliminates the issues related to memory management in real-time, leaving Servlets to be Robust.

Followed by the features of Java Servlets, we will dive into the Servlet Request and Response Phases.

Java Servlet Request

The Java Servlets operate in the form of Requests and Responses. The first phase is the request from the user's end. The HttpRequest object represents the HTTP request to a browser that the user sends to the web application. Thus, anything the browser sends is accessible through the HttpRequest.

Now, we will learn about the various segments of a request received by the Servlet.

1. HTTP Request Header
2. HTTP Request Parameters
3. HTTP Request InputStream
4. HTTP Request Context
5. HTTP Request Session

1. HTTP Request Header

The request headers are key-value pairs sent as a request by the browser, along with the HTTP request. The request headers contain information about what browser software is being used, what file types the browser can receive.

2. HTTP Request Parameters

Along with the request, the browser shares a few parameters related to the information required. Request parameters are technically a crucial part of the URL of an HTTP request.

3. HTTP Request InputStream

HTTP Request InputStream is an ordered stream of input data. The HTTP Request InputStream extends the java.io.InputStream library.

4. HTTP Request Context

We dedicate the HTTP Request Context to storing the metadata related to the application.

5. HTTP Request Session

The session object holds information about a specific user in between multiple requests. In case you set a new object during the session object request, the object will get available for you to access during any subsequent requests within the same session time scope.

Now, we will advance and learn the Java Servlet Response Phase.

Here's How to Land a Top Software Developer Job

Full Stack Development-MEAN

EXPLORE PROGRAM

Java Servlet Response

The second important part is the Response phase. The HttpResponse object is used to represent the HTTP response to your request. A web application sends back a response page to the user to respond to the HTTP request from the browser sent to your web application.

Let us now understand the parts of the Response Phase in the Java Servlets as follows.

1. HTTP Response Header
2. HTTP Response Content Type

3. HTTP Response Content Length

4. HTTP Response Write HTML

5. HTTP Response Redirection

1. HTTP Response Header

Similar to HTTP Request, the HTTP response includes headers. These are responsible for all the data written in the response.

2. HTTP Response Content Type

The Content-Type header tells the browser about the type of content you are sending in return.

3. HTTP Response Content-Length

The Content-Length header tells you about the number of bytes your Servlet is sending in return.

4. HTTP Response Write HTML

To send the HTML back to the browser, you need to retrieve the PrintWriter from the HTTP response object.

5. HTTP Response Redirection

Redirecting will help you turn yourself to a different webpage from the current one.

With both the Request and Response phases discussed, we will dive into the practical mode where we learn to set up the environment for Java Servlets and execute a sample program for better understanding.